Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Codage des nombres entiers relatifs, algorithme du complément à deux, arithmétique binaire et détection du dépassement de capacité (Overflow)

Synthèse exécutive : Ce cours traite de la représentation matérielle des entiers signés et non signés en mémoire binaire sur un nombre fixe de bits. Face à l'impossibilité physique des composants électroniques (transistors, bascules) d'enregistrer des signes '+' ou '-', l'architecture informatique utilise la convention du complément à deux. Cette méthode élimine le problème du double zéro présent dans la représentation 'Signe + Magnitude' et unifie l'addition et la soustraction au sein du même circuit d'additionneur dans l'Unité Arithmétique et Logique (ALU). Le cours détaille la procédure de conversion pas à pas, la gestion de la retenue sortante (Carry out) et les règles strictes de détection des erreurs de dépassement de capacité (Overflow).

1. Problématique matérielle et justification du Complément à Deux

Au niveau le plus bas de l'architecture matérielle, la mémoire et les registres processeur sont constitués de bascules électroniques incapables de stocker directement un signe '+' ou '-'. Pour représenter des nombres négatifs, plusieurs conventions ont été développées. La première idée, dite 'Signe et Magnitude' (où le bit de poids fort MSB indique le signe et les bits suivants la valeur absolue), comporte deux vices majeurs : elle génère deux codages distincts pour le nombre zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison et ralentit les tests d'égalité, et elle exige de fabriquer des circuits électroniques séparés pour l'addition et la soustraction. La convention du complément à deux résout intégralement ces limites : le zéro possède un codage unique (0000...0) et la soustraction A - B s'effectue physiquement comme l'addition A + (-B) au sein de la même Unité Arithmétique et Logique (ALU).

- Absence de signe physique : les transistors et bascules bistables ne stockent que deux états logiques (0 pour 0V, 1 pour Vcc).
- Échec du codage Signe + Magnitude : présence d'un double zéro (+0 et -0) gaspillant une combinaison binaire et complexifiant l'ALU.
- Unification par le complément à deux : la soustraction A - B est calculée sous la forme d'une addition A + (-B) en réutilisant le même circuit additionneur.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et qu'on effectue un kilomètre en marche arrière, le compteur repasse à 9999. En binaire sur 8 bits, faire 00000000 - 1 donne 11111111_2, qui définit la représentation exacte de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

Pour représenter un entier négatif -X en complément à deux sur un mot mémoire de N bits, l'algorithme doit suivre trois étapes indissociables : 1. Représenter la valeur absolue X en binaire naturel en le complétant impérativement par des 0 à gauche pour atteindre exactement N bits. 2. Inverser la totalité des bits (transformer les 0 en 1 et les 1 en 0), ce qui correspond au complément à un. 3. Ajouter 1 au résultat binaire obtenu. La plage des valeurs représentables sur N bits n'est pas symétrique : sur 8 bits, elle s'étend de -128 à +127. Cette dissymétrie s'explique par le fait que la combinaison zéro (00000000) a un MSB égal à 0 et appartient donc à la plage des nombres positifs.

- Algorithme des 3 étapes : Valeur absolue sur N bits -> Inversion de tous les bits (Complément à 1) -> Addition de 1 (Complément à 2).
- Bit de poids fort (MSB - Most Significant Bit) : indique le signe (0 pour les positifs, 1 pour les négatifs).
- Plage de valeurs dissymétrique : s'étend de -2^(N-1) à +(2^(N-1) - 1) sur N bits.

Cas concret enseignant : *Piège classique à éviter absolument en contrôle/partiel : il faut IMPÉRATIVEMENT fixer la taille N bits (par exemple 8 bits) AVANT d'effectuer l'inversion. Inverser le nombre '1100' sans écrire préalablement les zéros de gauche ('00001100') donne un résultat complètement faux !*

3. Arithmétique, Retenue (Carry) et Détection du Dépassement de Capacité (Overflow)

Les registres processeur travaillent sur une largeur fixe (8, 16, 32 ou 64 bits). Il est fondamental de ne pas confondre la retenue sortante (Carry Out) et le drapeau de dépassement de capacité (Overflow - OV). Lors d'un calcul en complément à deux, la retenue tout à gauche dépassent le N-ième bit est rejetée car le matériel effectue une arithmétique modulo 2^N. En revanche, un Overflow se produit lorsque le résultat mathématique excède la plage autorisée (-2^(N-1) à +2^(N-1)-1). En examen, la règle formelle de vérification d'un Overflow est la suivante : l'Overflow se produit si et seulement si deux nombres de même signe (deux positifs ou deux négatifs) sont additionnés et donnent un résultat de signe inverse. L'addition de deux nombres de signes différents ne peut jamais générer d'Overflow.

- Soustraction unifiée : calculée directement via A + (-B).
- Distinction Carry vs Overflow : la retenue sortante (Carry Out du bit MSB) est ignorée en complément à deux car le calcul est modulo 2^N, tandis que l'Overflow indique une erreur d'amplitude arithmétique.
- Règle d'or de détection d'Overflow : un dépassement survient si et seulement si l'addition de deux opérandes de même signe produit un résultat de signe opposé.

Cas concret enseignant : *L'anecdote historique du crash du vol 501 de la fusée Ariane 5 en 1996 : un nombre flottant sur 64 bits a été converti en entier signé sur 16 bits sans mécanisme de protection contre l'Overflow. Le dépassement de capacité non géré a corrompu les données du système de guidage, provoquant l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : En complément à deux sur N bits, l'arithmétique s'effectue modulo 2^N. Les registres ont une taille physique fixe de N bits : le bit de retenue qui déborde à la position N n'a aucun emplacement physique pour être stocké dans le registre de résultat. Mathématiquement, jeter cette retenue revient exactement à effectuer le calcul modulo 2^N, ce qui donne la valeur exacte recherchée.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait aucune différence au niveau des bits stockés dans le registre : la séquence 11111111 est strictement identique en mémoire. C'est le type de la donnée défini au niveau logiciel (ex: unsigned char vs signed char en C) et les instructions assemblées utilisées (ex: lecture des drapeaux de condition Carry ou Overflow) qui dictent la manière dont le processeur interprète et affiche la valeur.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des opérandes, effectuer l'addition binaire correspondante, donner le résultat décimal et indiquer s'il y a eu un Overflow.

Résolution pas à pas :
1. Conversion de +35 sur 8 bits : 35 = 32 + 2 + 1 = 00100011_2. 2. Conversion de -75 en complément à deux sur 8 bits : - Valeur absolue +75 sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (-75 décimal). 3. Pose de l'addition (+35) + (-75) : 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 4. Interprétation du résultat : Le bit MSB est à 1, le résultat est donc négatif. Pour retrouver sa valeur absolue, on applique le complément à deux sur 11011000_2 : - Inversion : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le résultat binaire 11011000_2 représente donc -40 en décimal. 5. Analyse de l'Overflow : On additionne un nombre

positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). L'addition de deux opérandes de signes différents ne peut jamais provoquer d'Overflow.

Résultat : Résultat binaire : 11011000_2 (-40 en décimal). Drapeau d'Overflow = 0 (aucun dépassement). Le calcul 35 - 75 = -40 est parfaitement valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage des entiers signés représentables sur un mot de 16 bits en complément à deux ?

R : La plage s'étend de -2^15 à +(2^15 - 1), c'est-à-dire de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors d'une addition d'entiers signés ?

R : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé.

Q : Quel est le résultat de l'application du complément à deux sur le nombre zéro (00000000_2) sur 8 bits ?

R : Inversion de 00000000_2 = 11111111_2. Ajout de 1 = 1 00000000_2. La retenue tout à gauche est rejetée sur 8 bits, ce qui redonne 00000000_2. Le zéro a un codage unique.